Lossless Client-Side Compression of Contract Uploads

An LZ77 and Huffman (DEFLATE) codec for the *Know What You Sign* contract-analysis system

Technical report, October 2026

Abstract

Know What You Sign helps non-expert signatories identify statistically unusual clauses in rental agreements, employment offers and terms of service. Its users are often on mobile data, and the contracts they hold are frequently scanned or photographed. Such PDFs can store every page as an uncompressed raster image: the project's three-page scanned test agreement is 16.7 MB, about 5.5 MB per page, so the server's 25 MB upload limit refused any scanned agreement longer than four or five pages. This report describes a lossless compression layer, implemented from first principles, that addresses the problem without the server ever storing the document.

The browser compresses the PDF before upload with a hand-written DEFLATE encoder (RFC 1951) that combines LZ77 dictionary matching over a 32 KB sliding window with hash chains and Huffman coding whose length-limited codes are built by the package-merge algorithm. Each 64 KB block is emitted either Huffman-coded or stored verbatim, whichever an exact bit-cost model favours by at least ten per cent. The stream is wrapped in a 16-byte container carrying the original length and a CRC-32. On the server, a hand-written table-driven decoder restores the file under a hard output cap, which defeats decompression bombs, and verifies its integrity before analysis begins.

Correctness is established against zlib as an independent oracle in both directions, and by a cross-language test in which the TypeScript encoder's output is decoded by the Python decoder. The scanned agreement is reduced from 16.7 MB to 294.5 KB, a ratio of 57:1, about 11 per cent smaller than zlib at its maximum level. Encoding takes 0.28 s and decoding 0.33 s. The effective size limit rises from about 5 to about 18 scanned pages.

Keywords: lossless compression, LZ77, Huffman coding, package-merge, DEFLATE, decompression bomb, client-side processing, privacy

1. Methodology

1.1 Characterising the data

The method was chosen after measuring the documents the system actually receives, not assumed in advance. Every object in each test PDF was inspected for its stream filter, and each file's compressibility was estimated with a general-purpose compressor. Table 1 shows that the two kinds of input behave very differently.

Document	Size	Internal encoding	gzip -9 ratio
Typed lease (sample_lease.pdf)	10.2 KB	Text, Flate-compressed streams	1.7 : 1
Typed terms of service (sample_tos.pdf)	7.6 KB	Text, Flate-compressed streams	1.6 : 1
Scanned lease (sample_lease_scanned.pdf)	16.7 MB	Raw 1405 × 1989 images, 8 bits per component, no filter	54.6 : 1

Table 1. Composition and compressibility of the test documents.

Two findings follow. First, the bytes are concentrated in scanned documents, whose page images are stored without any filter and consist largely of long runs of near-identical background pixels: exactly the redundancy that dictionary coding removes. Second, typed PDFs gain comparatively little because their content streams are already compressed. A useful method must therefore be very effective on raster redundancy, and must also cost almost nothing when there is little to gain.

1.2 Design constraints

- **Privacy.** An uploaded agreement contains names, addresses, salaries and identity numbers. The server writes it only to a temporary file deleted when the request ends. Compression must therefore shorten the transfer, not create stored copies, which places the encoder in the browser and the decoder in the request path.
- **Untrusted input.** The server decodes data supplied by strangers, so every resource the decoder consumes must be bounded before it is consumed.
- **Lossless.** The analysis reads clause text and figures from the PDF, and OCR reads scanned pixels. Any loss could alter a reported finding, so only lossless coding is admissible.
- **Compatibility.** Plain PDF uploads from the command line and other clients must continue to work through the same endpoint, unchanged.
- **Verifiability.** Every component must be checkable against an independent reference implementation, not only against itself.

1.3 System architecture

Figure 1 shows the end-to-end path. Compression runs in a Web Worker so that packing a large scan does not block the interface. The server recognises a compressed upload by its magic number, restores it, and from there on runs the existing pipeline unchanged.

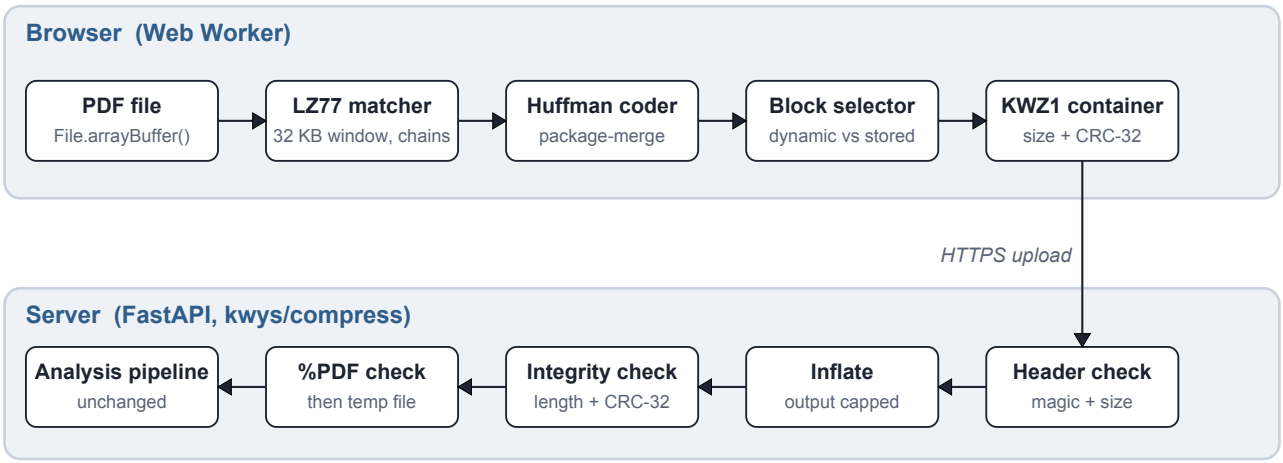


Figure 1. Upload path: encoding in the browser, bounded decoding and verification on the server.

1.4 Container format

Raw DEFLATE has no length or checksum, so it is framed in a 16-byte container named KWZ1 (Table 2). The magic number cannot begin a valid PDF, whose first bytes are always `%PDF`, so one endpoint can accept both forms without an extra request field.

Offset	Size	Field	Purpose
0	4	Magic `KWZ1`	Distinguishes a compressed upload from a plain PDF
4	4	Original size (LE)	Rejects oversized files before any decoding work
8	4	CRC-32 of original (LE)	Detects corruption or tampering after decoding
12	4	Reserved (zero)	Room for a future format revision
16	n	Raw DEFLATE stream	The compressed PDF (RFC 1951)

Table 2. The KWZ1 container.

Emitting standard DEFLATE rather than an invented bitstream was deliberate: it makes zlib available as an independent oracle for both halves of the codec (Section 1.7).

1.5 Encoder

1.5.1 Block segmentation

The input is divided into 64 KB blocks, and each block receives its own Huffman codes. This lets the code adapt as a PDF moves between text streams, already-compressed objects and raw image data. The LZ77 state persists across block boundaries, so a match may refer back into earlier blocks, whichever way those blocks were emitted.

1.5.2 LZ77 dictionary matching

Each position is hashed on its next three bytes into a 2^{15}-entry head table. A `prev` array, indexed by position modulo the window, links every position to the previous one with the same hash. The resulting chain lists earlier occurrences of the same three-byte prefix, newest first. The matcher walks the chain, keeps the longest match, and stops early at the maximum length. Before a full comparison, it tests the single byte that would extend the current best match, which rejects most candidates in one comparison. Matching is greedy: the longest match at the current position is taken and every position it covers is inserted into the hash table. Table 3 gives the parameters.

Parameter	Value	Rationale
Window (maximum distance)	32 768 bytes	DEFLATE's limit
Match length	3 to 258 bytes	DEFLATE's limits
Hash table size	2^{15} buckets	One per window position
Maximum chain length	64 candidates	Bounds the work per position, trading a little ratio for predictable speed
Block size	65 536 bytes	Adaptivity versus per-block header overhead

Table 3. LZ77 parameters.

A match is stored internally as one 32-bit token, (distance << 9) | length. Because every distance is at least 1, a match token always exceeds 255, so literals and matches share one array without a tag bit.

1.5.3 Huffman coding

Each block's tokens are mapped to DEFLATE's two alphabets: 286 literal/length symbols, in which symbol 256 marks the end of the block, and 30 distance symbols. Values within a symbol's range are sent as raw extra bits. Symbol frequencies are counted per block.

Length-limited codes by package-merge. An unconstrained Huffman code can assign codes longer than DEFLATE permits (15 bits, or 7 bits for the code-length alphabet). Instead of building a Huffman tree and repairing it heuristically, the encoder uses the package-merge algorithm (Larmore and Hirschberg, 1990), which yields the optimal prefix code subject to a maximum length L directly. Symbols sorted by weight are repeatedly paired into packages and merged back with the original symbols, L − 1 times. The

first 2n − 2 items are selected, and each symbol's code length is the number of selected items containing it. The result satisfies the Kraft equality exactly:

$$\Sigma_i 2^{-l_i} = 1, \quad \text{with } 1 \le l_i \le L$$

Equality, not just inequality, matters: zlib and other strict decoders reject an incomplete code. A degenerate alphabet with one symbol, or none, is padded to two symbols of length 1, so that the code is always complete.

Canonical codes. Only code lengths are transmitted. Both sides derive identical codes by assigning consecutive values in order of (length, symbol). Because DEFLATE packs bits least-significant first but defines Huffman codes most-significant first, each code is bit-reversed once when it is built, so the bit writer can emit it directly.

Header compression. The block's code lengths are run-length encoded with DEFLATE's three repeat symbols: 16 repeats the previous length 3 to 6 times, 17 sends 3 to 10 zeros, and 18 sends 11 to 138 zeros. The run-length symbols are then coded with a third, 19-symbol Huffman code limited to 7 bits, whose lengths are sent in DEFLATE's fixed permutation order with trailing zeros dropped.

1.5.4 Choosing each block's representation

For every block, the encoder computes the exact size of its dynamic-Huffman encoding: header, code-length code, and the frequency-weighted code and extra-bit lengths of every symbol. It compares this with the size of storing the block verbatim, which costs its bytes plus 40 bits of framing per 65 535-byte stored segment. The block is Huffman-coded only if

$$C_{\text{dynamic}} \le 0.9 \times C_{\text{stored}}$$

and is stored otherwise. The ten per cent margin is not about compression ratio, since the cheaper option would always be smaller. It is about decoding cost. A stored block costs the server a single memory copy, whereas a block of mostly literals costs a Python-level operation per byte. A marginal saving of a few per cent is not worth that work, so already-compressed regions of a PDF pass through at almost no cost.

1.5.5 Whole-file decision and failure handling

The same rule is applied to the file as a whole: the compressed container is sent only if it is at most 90% of the original size. Otherwise the untouched PDF is sent. Compression is treated strictly as an optimisation, so any failure in the worker also falls back to sending the original file. After compression, the browser checks the 25 MB upload limit against the bytes it will actually send. Before compression, it checks a 100 MB limit against the PDF itself.

1.6 Decoder

The server-side decoder is a complete RFC 1951 implementation in Python, covering stored, fixed-Huffman and dynamic-Huffman blocks, so it accepts any conforming encoder and not only this one.

- **Table-driven Huffman decoding.** Each code is expanded into a single lookup table of 2^{maxlen} entries indexed by the next *maxlen* input bits. Every entry whose low bits equal a reversed code holds that code's symbol and length. One lookup therefore decodes one symbol, whatever bits follow it.
- **Validation of codes.** Before a table is built, the code lengths are checked against the Kraft inequality. An over-subscribed code, which no prefix code can realise, is rejected as corrupt.
- **Overlapping copies.** When a back-reference's distance is shorter than its length, as in “repeat the last byte 258 times”, the source is a repeating pattern. The decoder copies that pattern in whole repetitions instead of byte by byte, which is what makes long runs of background pixels cheap to restore.
- **Resource bounds.** A few kilobytes of DEFLATE can legitimately describe gigabytes of output. The declared size is checked against the 100 MB limit before decoding, and the decoder is then capped at the declared size and checks the cap *before* writing each literal and each copy. Memory is never spent beyond the limit, whether the header is honest or not.

- **Integrity.** After decoding, the output length and CRC-32 must both equal the header's values, and the result must still begin with `%PDF`. Failures map to distinct responses: HTTP 413 when the file is too large uncompressed, and HTTP 400 when the upload is damaged.

1.7 Evaluation methodology

1.7.1 Correctness

A self-consistent encoder and decoder can share one mistake and still round-trip each other. Each half was therefore verified against zlib as an independent reference:

- **Encoder:** every output must be decoded by zlib's `inflateRawSync` to exactly the original bytes. Inputs: the empty input, one and two bytes, a 1 MB run of zeros (overlapping copies), short-period text, 300 KB of pseudo-random bytes (all stored blocks), mixed inputs whose matches cross block boundaries, and all three test PDFs. The CRC-32 is checked against zlib's.
- **Decoder:** must reproduce inputs compressed by zlib at levels 0, 1, 6 and 9, and in forced fixed-Huffman mode, across seven inputs that include empty, random, text-like and real PDF data.
- **Cross-language:** each test PDF is compressed by the production TypeScript encoder, run under Node, and must be restored exactly by the production Python decoder.
- **System:** a compressed upload through the HTTP API must produce exactly the same findings as the plain PDF.

1.7.2 Robustness

Adversarial inputs were tested: a decompression bomb (10 MB of zeros, under 20 KB compressed) against a 1 MB cap; a literal-only stream whose 1-bit codes expand each input byte eightfold; headers that overstate or understate the true size; a wrong checksum; streams truncated at three points; and random bytes after a valid header. Each must be refused with the right error before excess memory is used.

1.7.3 Performance protocol and metrics

Compression ratio is reported as original size divided by compressed size, and space saving as one minus the inverse. Timings are the best of five runs on an Intel Core i9-9980HK at 2.4 GHz: the encoder under Node 22, which uses the same V8 engine as Chrome, and the decoder under CPython. The baseline is zlib at its maximum level (9). Table 4 summarises the measurements.

Document	Original	KWZ1 upload	Ratio	zlib -9	Encode	Decode
Scanned lease	16 771 354 B	294 501 B	57.0 : 1	331 732 B	283 ms	329 ms
Typed lease	10 459 B	6 294 B	1.66 : 1	6 198 B	2.9 ms	3.7 ms
Typed terms of service	7 818 B	5 020 B	1.56 : 1	4 937 B	1.5 ms	2.4 ms

Table 4. Measured results. KWZ1 sizes include the 16-byte header; zlib sizes are raw DEFLATE.

On the scanned agreement, the encoder is about 11% smaller than zlib at level 9, a space saving of 98.2%. On the small typed PDFs it is within 2% of zlib, and both typed files clear the 90% threshold, so they are also sent compressed. On general text such as the harvested judgment corpus and HTML pages, it stayed within 3 to 6% of zlib. All 244 Python tests and 6 encoder tests pass, and the continuous-integration workflow runs the cross-language test on every push.

Limitation. In pure Python the decoder handles literal-dominated data at roughly 0.4 s per megabyte of output. The stored-block rule keeps ordinary PDFs away from this path, but a deliberately constructed 25 MB upload could occupy the server for roughly 15 to 20 s. That is within the cost an attacker can already impose by uploading a large scan for OCR, but it is the natural target for future optimisation.